Matriz de Permisos y Control de Acceso (RBAC) — GestionExamio

> Documento de arquitectura que define la política de control de acceso basada en roles (RBAC) de la API. Sirve como **Fuente de Verdad** para la implementación de guardias en Frontend y validaciones en Backend.

---

1. Introducción y Arquitectura de Seguridad

1.1 Modelo de Autenticación

El sistema utiliza **autenticación stateless** basada en JWT (JSON Web Token). No hay sesiones en servidor ni cookies de autenticación. Cada petición debe incluir el token en el header `Authorization: Bearer <token>`.

Flujo de login:

```
POST /api/auth/login { Email, Password }
       │
       ▼
  auth.service.findUserByEmail(Email)
       │ WHERE U.Activo = 1
       ▼
  bcrypt.compare(Password, user.PasswordHash)
       │
       ├── Éxito → jwt.sign({ id, email, role, idAlumno, idDocente }, SECRET, { expiresIn: '1h' })
       │            → Response 200 { token, user: { id, email, role, name, identificadorInstitucional } }
       │
       └── Fallo → Response 401 { message: "Credenciales inválidas." }
```

Forma del payload JWT:

```
{
  "id": 1,
  "email": "lfernandez@cervantes.edu.ar",
  "role": "docente",
  "idAlumno": null,
  "idDocente": 100542,
  "iat": 1718611200,
  "exp": 1718614800
}
```

Claim	Tipo	Descripción
`id`	`number`	ID del registro en `Usuario`

Claim	Tipo	Descripción
`email`	`string`	Email del usuario (unique)
`role`	`'admin' | 'docente' | 'alumno'`	Rol del usuario
`idAlumno`	`number | null`	Matrícula del alumno (solo si `role === 'alumno'`)
`idDocente`	`number | null`	Legajo del docente (solo si `role === 'docente'`)
`iat`	`number`	Timestamp de emisión
`exp`	`number`	Timestamp de expiración (1 hora)

> **Regla de BD:** La tabla `Usuario` tiene un CHECK que asegura consistencia: si `Rol = 'alumno'` entonces `IdAlumno IS NOT NULL AND IdDocente IS NULL`; si `Rol = 'docente'` entonces `IdDocente IS NOT NULL AND IdAlumno IS NULL`; si `Rol = 'admin'` ambos son NULL.

1.2 Flujo de Middlewares

Cada petición protegida atraviesa dos capas antes del controlador:



```
│  - Existencia de recurso (404)   │
│  → Operación BD                  │
│  → Response 200 / 500            │
└──────────────────────────────────┘
```

> **Nota:** El middleware `validateRequest` es opcional. Solo se aplica en endpoints que reciben body (`POST`, `PUT`). Endpoints públicos (`GET /api/carreras`, etc.) no pasan por ningún middleware.

1.3 Mapa de Roles

Rol	¿Quién lo tiene?	Acceso general
`admin`	Administradores del sistema	**Acceso total.** CRUD de todas las entidades, dashboard analítico, gestión de usuarios.
`docente`	Profesores	Acceso a alumnos, comisiones, evaluaciones, calificación y corrección. **Sin** acceso a gestión de usuarios ni dashboard admin.
`alumno`	Estudiantes	Solo sus propios datos e intentos. No puede crear/modificar recursos del sistema.

---

2. Matriz Global de Permisos (RBAC)

2.1 Módulo Auth

Endpoint	alumno	docente	admin
`POST /api/auth/login`	✅ Público	✅ Público	✅ Público

2.2 Módulo Alumnos

Endpoint	alumno	docente	admin
`GET /api/alumnos`	❌	✅	✅
`GET /api/alumnos/:matricula`	✅ (propio)	✅	✅
`POST /api/alumnos`	❌	❌	✅
`PUT /api/alumnos/:matricula`	❌	❌	✅
`DELETE /api/alumnos/:matricula`	❌	❌	✅
`GET /api/alumnos/:matricula/materias`	✅ (propio)	✅	✅
`GET /api/alumnos/:matricula/estadisticas`	✅ (propio)	✅	✅

> ⚠️ **Ausencia de guardia de titularidad:** Los endpoints marcados como ✅ `(propio)` permiten al rol `alumno` acceder, pero deben validar que `:matricula === req.user.idAlumno`. (ver §3.1).

2.3 Módulo Docentes

Endpoint	alumno	docente	admin
`GET /api/docentes`	❌	✅	✅
`GET /api/docentes/:legajo`	❌	✅	✅
`POST /api/docentes`	❌	❌	✅
`PUT /api/docentes/:legajo`	❌	❌	✅
`DELETE /api/docentes/:legajo`	❌	❌	✅
`GET /api/docentes/:legajo/comisiones`	❌	✅	✅
`GET /api/docentes/estado/:estado`	❌	❌	✅
`POST /api/docentes/:legajo/asignar`	❌	❌	✅
`GET /api/docentes/:legajo/materias`	❌	✅	✅
`GET /api/docentes/:legajo/evaluaciones`	❌	✅	✅

2.4 Módulo Carreras

Endpoint	alumno	docente	admin
`GET /api/carreras`	✅ Público	✅ Público	✅ Público
`GET /api/carreras/:id`	✅ Público	✅ Público	✅ Público
`POST /api/carreras`	✅ Público	✅ Público	✅ Público
`PUT /api/carreras/:id`	✅ Público	✅ Público	✅ Público
`DELETE /api/carreras/:id`	✅ Público	✅ Público	✅ Público
`GET /api/carreras/:id/plan`	✅ Público	✅ Público	✅ Público
`GET /api/carreras/:id/alumnos`	✅ Público	✅ Público	✅ Público

> ⚠️ **Endpoints públicos sin autenticación:** Carreras, Materias, Comisiones y Divisiones (GET) son de acceso público. Cualquier persona con la URL puede consultarlos.

2.5 Módulo Materias

Endpoint	alumno	docente	admin
`GET /api/materias`	✅ Público	✅ Público	✅ Público
`GET /api/materias/:id`	✅ Público	✅ Público	✅ Público
`POST /api/materias`	✅ Público	✅ Público	✅ Público
`PUT /api/materias/:id`	✅ Público	✅ Público	✅ Público
`DELETE /api/materias/:id`	✅ Público	✅ Público	✅ Público
`GET /api/materias/:id/alumnos`	✅ Público	✅ Público	✅ Público

Endpoint	alumno	docente	admin
`POST /api/materias/:id/division`	✅ Público	✅ Público	✅ Público
`GET /api/materias/:id/comisiones`	✅ Público	✅ Público	✅ Público

2.6 Módulo Comisiones

Endpoint	alumno	docente	admin
`GET /api/comisiones`	✅ Público	✅ Público	✅ Público
`GET /api/comisiones/:id/alumnos`	✅ Público	✅ Público	✅ Público

2.7 Módulo Divisiones

Endpoint	alumno	docente	admin
`GET /api/divisiones`	✅ Público	✅ Público	✅ Público
`GET /api/divisiones/:id`	✅ Público	✅ Público	✅ Público
`POST /api/divisiones`	❌	❌	✅
`PUT /api/divisiones/:id`	❌	❌	✅ (501)
`DELETE /api/divisiones/:id`	❌	❌	✅ (501)

2.8 Módulo Evaluaciones

Endpoint	alumno	docente	admin
`POST /api/evaluaciones/guardar-completo`	❌	✅	✅
`PUT /api/evaluaciones/:id/estado`	❌	✅	✅
`GET /api/evaluaciones/intento/:idIntento/renderizar`	✅	✅	✅
`GET /api/evaluaciones/pregunta/:idPregunta/opciones/:idIntento`	✅	✅	✅
`GET /api/evaluaciones/publicacion/:id`	✅	✅	✅
`GET /api/evaluaciones/pendientes/:matricula`	✅ (propio)	✅	✅
`GET /api/evaluaciones/:id/detalle`	❌	✅	✅
`GET /api/evaluaciones/:id/alumnos`	❌	✅	✅

2.9 Módulo Intentos

Endpoint	alumno	docente	admin
`POST /api/intentos/iniciar`	✅	❌	✅

Endpoint	alumno	docente	admin
`POST /api/intentos/guardar-respuesta`	✅	❌	✅
`POST /api/intentos/:id/finalizar`	✅	❌	✅
`GET /api/intentos/alumno/:idAlumno`	✅ (propio)	✅	✅
`GET /api/intentos/alumno/:idAlumno/detallados`	✅ (propio)	✅	✅
`GET /api/intentos/alumno/:idAlumno/historial-corregido`	✅ (propio)	✅	✅
`GET /api/intentos/:idIntento/completo`	✅ (propio + guardia)	✅	✅
`PUT /api/intentos/:idIntento/calificar`	❌	✅	✅
`PUT /api/intentos/:idIntento/preguntas/:idPregunta/corregir`	❌	✅	❌

> 📌 **Endpoints de escritura exclusivos para `alumno`:** `iniciar`, `guardar-respuesta` y `finalizar` están restringidos a `alumno` y `admin` — el `docente` **no** puede iniciar ni responder exámenes en nombre de un alumno.
>
> 📌 **Corregir vs Calificar:** El endpoint `corregir` (corrección manual de preguntas abiertas) es **exclusivo de `docente`**. El endpoint `calificar` (asignación de `NotaFinal`) permite tanto a `admin` como a `docente`.

2.10 Módulo Telemetría

Endpoint	alumno	docente	admin
`POST /api/telemetria/alerta`	✅	❌	✅
`GET /api/telemetria/alerta/:idIntento`	❌	✅	✅
`POST /api/telemetria/navegacion`	✅	❌	✅

> 📌 El `docente` puede **leer** alertas de fraude pero **no puede registrarlas** ni registrar navegación. Esto evita que un docente inyecte alertas falsas.

2.11 Módulo Actas

Endpoint	alumno	docente	admin
`GET /api/actas`	❌	✅	✅
`GET /api/actas/:id`	❌	✅	✅
`PUT /api/actas/:id/cerrar`	❌	✅	✅
`POST /api/actas/crear-cierre`	❌	✅	✅
`POST /api/actas/cargar-detalle`	❌	✅	✅

> 📌 El `alumno` no tiene acceso a ningún endpoint del módulo Actas. Las actas son responsabilidad exclusiva de docentes y administradores.

2.12 Módulo Usuarios

Endpoint	alumno	docente	admin
`GET /api/usuarios`	❌	❌	✅
`POST /api/usuarios`	❌	❌	✅
`PUT /api/usuarios/:id`	❌	❌	✅
`DELETE /api/usuarios/:id`	❌	❌	✅

> 📌 Gestión de usuarios es **exclusiva de** `admin`. Ni docentes ni alumnos pueden crear, modificar ni listar cuentas de usuario.

2.13 Módulo Admin — Dashboard Analítico

Endpoint	alumno	docente	admin
`GET /api/admin/dashboard/analitica`	❌	❌	✅

---

3. Guardias de Titularidad y Contexto

3.1 ¿Qué es una guardia de titularidad?

El `checkRole` permite o deniega el acceso según el **rol** del usuario. Pero dentro de un mismo rol, puede haber restricciones adicionales: la **pertenencia del dato**.

Ejemplo: dos alumnos (`idAlumno: 30500` y `idAlumno: 30999`) pueden ambos acceder a `GET /api/intentos/alumno/:idAlumno` porque su rol `alumno` está autorizado. Pero alumno `30500` **no debería** poder ver los intentos de alumno `30999`.

3.2 Guardias implementadas actualmente

Ubicación	Guardia	Comportamiento
`intentos.controller.ts:203`	**Radiografía de intento**	Si `role === 'alumno'`, verifica que `intento.IdAlumno === req.user.idAlumno`. Si no coincide → `403`.
`intentos.controller.ts:30`	**Límite de intentos**	Antes de iniciar, cuenta `IntentosRealizados` y compara con `CantidadIntentosMaximos`. Si se excedió → `403`.
`intentos.controller.ts:289`	**Estado para calificar**	Si `Estado === 'en_progreso'` → `400`. No se puede calificar un examen en curso.
`intentos.controller.ts:321`	**Estado para corregir**	Si `Estado === 'en_progreso'` → `400`. No se puede corregir un examen en curso.

3.3 Brechas de titularidad identificadas

Los siguientes endpoints **permiten** al rol `alumno` acceder a datos de **cualquier** alumno, porque solo validan el rol pero NO cruzan el `:idAlumno`/`:matricula` del path param contra el `req.user.idAlumno` del token:

Endpoint	Problema
`GET /api/intentos/alumno/:idAlumno`	Alumno A puede ver historial de Alumno B cambiando `:idAlumno`
`GET /api/intentos/alumno/:idAlumno/detallados`	Alumno A puede ver detallados de Alumno B
`GET /api/intentos/alumno/:idAlumno/historial-corregido`	Alumno A puede ver notas de Alumno B
`GET /api/alumnos/:matricula`	Alumno A puede consultar datos personales de Alumno B
`GET /api/alumnos/:matricula/materias`	Alumno A puede ver las materias de Alumno B
`GET /api/alumnos/:matricula/estadisticas`	Alumno A puede ver estadísticas de Alumno B
`GET /api/evaluaciones/pendientes/:matricula`	Alumno A puede ver las evaluaciones pendientes de Alumno B

Recomendación: Implementar un middleware o helper reutilizable (ej. `checkOwnership('IdAlumno')`) que compare el path param con `req.user.idAlumno` cuando el role sea `alumno`.

```
// Ejemplo de implementación
export const checkOwnership = (paramName: string) => {
  return (req: Request, res: Response, next: NextFunction) => {
    if ((req as any).user.role === 'alumno') {
      const paramValue = parseInt(req.params[paramName], 10);
      if (paramValue !== (req as any).user.idAlumno) {
        return res.status(403).json({ ... });
      }
    }
    next();
  };
};
```

3.4 Casos especiales de titularidad

Alumno — Solo sus propios intentos

El alumno puede iniciar, guardar y finalizar intentos (`POST /api/intentos/iniciar`, `/guardar-respuesta`, `/:id/finalizar`). Estos endpoints no tienen guardia de titularidad explícita porque:

- `iniciar` recibe `IdAlumno` en el **body** (no en el token).
- `guardar-respuesta` referencia a `IdIntento` (no al alumno directamente).
- `finalizar` referencia a `IdIntento` por params.

> **Riesgo:** Un alumno podría enviar `IdAlumno: 30999` en el body de `POST /api/intentos/iniciar` e iniciar un examen en nombre de otro alumno. Se recomienda sobrescribir `IdAlumno` con `req.user.idAlumno` en el controlador en lugar de leerlo del body.

Docente — Acceso contextual a evaluaciones

El docente accede a evaluaciones a través de los endpoints de su módulo (`GET /api/docentes/:legajo/evaluaciones`), que filtran por `Legajo` usando el parámetro de ruta. No hay una guardia que compare contra `req.user.idDocente`. Un docente podría consultar evaluaciones de otro docente.

> **Riesgo:** Bajo, porque los SPs internos filtran por `MateriaDivision.IdDocente` y los docentes solo ven sus propias comisiones. El riesgo es principalmente informativo (ver el listado de evaluaciones de otro docente sin poder interactuar con ellas).

Corrección / Calificación — Restricciones de escritura

- **Calificar** (`PUT /api/intentos/:idIntento/calificar`): Solo roles `admin` y `docente`. Guardia adicional: el intento no debe estar `en_progreso`.
- **Corregir** (`PUT /api/intentos/:idIntento/preguntas/:idPregunta/corregir`): **Solo** rol `docente`. Ni siquiera `admin` puede corregir preguntas manualmente.
- La corrección manual solo afecta a `Respuesta.EsCorrecta` y `Respuesta.ComentariosDocente`. **No altera** las opciones de mc/vf que se auto-corrigieron al momento de la entrega.

---

4. Informe de Comportamiento ante Fallos de Acceso

4.1 Mapa de Códigos HTTP

Código	Causa	Body de respuesta	¿Quién lo recibe?
401	Token ausente	`{ "success": false, "message": "Acceso no autorizado" }`	Cualquier petición sin `Authorization` header
401	Token inválido o expirado	`{ "success": false, "message": "Token inválido" }`	Cualquier petición con token corrupto/expirado
401	Credenciales incorrectas	`{ "success": false, "message": "Credenciales inválidas." }`	Login con email/contraseña incorrectos
401	Usuario inactivo	`{ "success": false, "message": "Credenciales inválidas." }`	Login de usuario con `Activo = 0` (mismo error genérico por seguridad)
403	Rol no autorizado	`{ "success": false, "message": "No tiene permisos para realizar esta acción" }`	Token válido pero el `role` no está en la whitelist del `checkRole`
403	Violación de titularidad	`{ "success": false, "message": "Acceso denegado. No tienes permisos para ver la radiografía de este intento." }`	Alumno intenta ver intento de otro alumno

Código	Causa	Body de respuesta	¿Quién lo recibe?
403	Límite de intentos agotado	`{ "success": false, "message": "Acceso denegado: Has alcanzado el límite máximo de intentos permitidos para esta evaluación." }`	Alumno intenta iniciar un examen sin intentos disponibles
400	Estado inválido (calificar)	`{ "success": false, "message": "No se puede calificar un examen que aún está en progreso." }`	Docente/admin intenta calificar un intento `en_progreso`
400	Estado inválido (corregir)	`{ "success": false, "message": "No se puede corregir un examen en progreso" }`	Docente intenta corregir un intento `en_progreso`
400	Validación Zod falla	`{ "success": false, "message": "Datos inválidos", "errors": [...] }`	Payload mal formado en cualquier `POST`/`PUT`
404	Recurso no encontrado	`{ "success": false, "message": "Intento no encontrado" }`	ID inexistente en cualquier endpoint
500	Error interno	`{ "success": false, "message": "<mensaje de error>" }`	Excepción no capturada o error de BD

4.2 Árbol de decisión de autorización

```
¿Tiene header Authorization?
├── No  → 401 "Acceso no autorizado"
└── Sí
    └── ¿jwt.verify(token) es exitoso?
        ├── No → 401 "Token inválido"
        └── Sí
            └── ¿decoded.role está en la whitelist del checkRole?
                ├── No → 403 "No tiene permisos para realizar esta acción"
                └── Sí
                    └── ¿Controlador tiene guardia de titularidad?
                        ├── No → Pasa a lógica de negocio
                        └── Sí
                            └── ¿El recurso pertenece al usuario?
                                ├── No → 403 "Acceso denegado..."
                                └── Sí → Pasa a lógica de negocio
```

4.3 Notas de seguridad

> **Credenciales genéricas (401):** El sistema **no** distingue entre "email no existe", "contraseña incorrecta" y "usuario inactivo". Siempre devuelve el mismo mensaje genérico para evitar enumeración de usuarios.

> **Inactividad vs Token expirado:** El token JWT expira a la hora. No hay mecanismo de refresh token. El Frontend debe redirigir al login cuando recibe un `401`.

> **Endpoints públicos (sin checkRole):** Los módulos Carreras, Materias y Comisiones (GET) no tienen autenticación. Cualquier persona con la URL puede leer estos catálogos.

---

5. Resumen de Brechas de Seguridad Detectadas

#	Brecha	Severidad	Endpoints afectados
1	**Falta de guardia de titularidad en rutas de alumnos** — un alumno puede acceder a datos de cualquier otro alumno cambiando `:idAlumno` o `:matricula` en la URL	🔴 Alta	`GET /api/intentos/alumno/:idAlumno*`, `GET /api/alumnos/:matricula*`, `GET /api/evaluaciones/pendientes/:matricula`
2	`IdAlumno` **en body no validado contra token** — en `POST /api/intentos/iniciar`, el alumno podría enviar `IdAlumno: 30999` e iniciar un examen en nombre de otro	🟡 Media	`POST /api/intentos/iniciar`
3	**Docente puede ver evaluaciones de otros docentes** — el endpoint `GET /api/docentes/:legajo/evaluaciones` no valida contra `req.user.idDocente`	🟢 Baja	`GET /api/docentes/:legajo/evaluaciones`
4	**Login sin validación Zod** — `POST /api/auth/login` no pasa por `validateRequest(LoginSchema)` (endpoint público)	🟡 Media	`POST /api/auth/login`

Documento generado a partir del análisis de `README.md`, `MAQUINA_ESTADOS.md` y el código fuente de rutas y controladores. Última actualización: 2026-06-17.